

GEETA UNIVERSITY · SUMMER INTERNSHIP 2026

Day 4 + Day 5

100 Questions — Error Handling, Modules,
DOM Manipulation & Events

100

Total

35

Predict Output

50

Code Karo

15

Mini-Project

Questions marked HARD are intentionally brutal.

Predict Output · Code Karo · Mini-Project

1

Yeh code kya print karega?

```
try { throw new Error('oops') } catch(e) {  
  console.log(e.message, e instanceof Error)  
} finally { console.log('done') }
```

PREDICT OUTPUT

DAY 4

2

Kya print hoga?

```
try { null.x } catch(e) { console.log(e.constructor.name) }
```

PREDICT OUTPUT

DAY 4

3

Kya print hoga aur kyun?

```
try {  
  try { throw 42 }  
  finally { console.log('inner') }  
} catch(e) { console.log('caught', e) }  
finally { console.log('outer') }
```

PREDICT OUTPUT

DAY 4

HARD

4

Kya print hoga?

```
function f() { try { return 1 } finally { return 2 } }  
console.log(f())
```

PREDICT OUTPUT

DAY 4

5

Kya print hoga?

```
async function go() {  
  try { await Promise.reject('bad') }  
  catch(e) { console.log(e) }  
  finally { console.log('fin') }  
}  
go()
```

PREDICT OUTPUT

DAY 4

HARD

6

Kya error type aayega?

```
try { undeclaredVar } catch(e) { console.log(e.constructor.name) }
```

PREDICT OUTPUT

DAY 4

7

Kva print hoga?

```
class AppError extends Error {  
  constructor(msg, code) { super(msg); this.code = code }  
}  
try { throw new AppError('fail', 404) }  
catch(e) { console.log(e.message, e.code, e instanceof Error) }
```

PREDICT OUTPUT

DAY 4

HARD

8

Kva print hoga?

```
try { JSON.parse('{bad}') } catch(e) { console.log(e instanceof SyntaxError) }
```

PREDICT OUTPUT

DAY 4

9

Kva print hoga — output order bhi batao:

```
Promise.resolve()  
  .then(() => { throw new Error('p') })  
  .catch(e => console.log('c', e.message));  
try { throw new Error('s') } catch(e) { console.log('s', e.message) }
```

PREDICT OUTPUT

DAY 4

HARD

10

Yeh code mentally run karo — error ka exact name batao:

```
const obj = {};  
obj.foo.bar;
```

PREDICT OUTPUT

DAY 4

11

ValidationError. NetworkError. aur AuthError class banao

Teeno Error se extend hon. Alag name aur extra property: field, statusCode, userId. Phir handle(err) function banao jo instanceof se sahi type detect kare.

CODE KARO

DAY 4

12

safeDiv(a, b) function banao

b===0 par custom DivisionByZeroError throw karo. Proper try/catch — error message console mein print ho.

CODE KARO

DAY 4

13

retrv(fn, times) — asvnc retrv function banao

fn ko times baar retry kare. Har attempt pe error log ho. Sab fail ho jaayein toh custom MaxRetriesError throw karo with attempt count.

CODE KARO

DAY 4

HARD

14

ES Modules project banao

mathUtils.js: add, sub, mul, div (named exports). div zero pe throw kare.
main.js mein sab import karke test karo.

CODE KARO

DAY 4

15

CommonJS loader module banao

logger.js mein log(level, msg) function. require se app.js mein import karo.
module.exports ka sahi use explain karo – default vs named.

CODE KARO

DAY 4

16

Dvnamic import() se plugin loader banao

loadPlugin(name) function jo runtime par ./plugins/\${name}.js load kare.
Plugin exist nahi kare toh custom PluginNotFoundError with plugin name.

CODE KARO

DAY 4

HARD

17

confia.is with default export banao

Object: { env, port, dbUrl }. Import karo, sab console karo.
Koi value missing ho toh descriptive error throw karo.

CODE KARO

DAY 4

18

errorBoundarv(fn) hiaher-order function banao

fn throw kare → { ok: false, error: e.message }
fn succeed kare → { ok: true, data: result }
Sync aur async dono handle kare.

CODE KARO

DAY 4

HARD

19

Teen utility modules + barrel export banao

stringUtils.js, arrayUtils.js, objectUtils.js banaao.
index.js se sab re-export karo. main.js mein index se import karo.

CODE KARO

DAY 4

20

fetchWithTimeout(url, ms) function banao

ms milliseconds ke baad custom TimeoutError throw kare.
Promise.race() use karo. Dono (fetch + timeout) ka proper cleanup karo.

CODE KARO

DAY 4

HARD

21

Custom EventEmitter class banao (Node wala nahi — apna)

on(event, cb), off(event, cb), emit(event, ...args) implement karo.
Listener throw kare toh baaki listeners rok mat do — sab run hone chahiye.

CODE KARO

DAY 4

HARD

22

parseJSON(str) safe wrapper banao

try/catch/finally use karo.
Valid JSON → parsed object. Invalid → null + error log.

CODE KARO

DAY 4

23

Kva print hoga?

```
const obj = { get val() { throw new Error('no') } };  
try { const { val } = obj }  
catch(e) { console.log(e.message) }
```

PREDICT OUTPUT

DAY 4

HARD

24

Kva print hoga?

```
function* gen() {  
  try { yield 1; yield 2 }  
  finally { yield 3 }  
}  
const g = gen();  
console.log(g.next(), g.return('done'), g.next())
```

PREDICT OUTPUT

DAY 4

25

Kva print hoga?

```
class E extends Error {}  
const e = new E('test');  
console.log(e instanceof E, e instanceof Error, e.name)
```

PREDICT OUTPUT

DAY 4

HARD

51

Top-level await — kab valid. kab error?

```
// file: main.js  
const m = await import('./mod.js');  
console.log(m.default, m.named);  
// Kab chalega? Kab SyntaxError?
```

PREDICT OUTPUT

DAY 4

HARD

52

Kva print hoga?

```
try { eval('{{bad syntax{{' }) }  
catch(e) { console.log(e.name) }
```

PREDICT OUTPUT

DAY 4

53

Kva print hoga?

```
class HttpError extends Error {  
  constructor(code) {  
    super('HTTP ' + code);  
    this.code = code;  
    this.name = 'HttpError';  
  }  
}  
  
const e = new HttpError(500);  
console.log(e.message, e.stack?.split('\n')[0])
```

PREDICT OUTPUT

DAY 4

HARD

54

Runtime behaviour mein kva farq hai?

```
import x from './a.js' // ESM  
const x = require('./a.js') // CJS  
// Execution order? Caching? Live binding?
```

PREDICT OUTPUT

DAY 4

55

Kva print hoga?

```
const err = new TypeError('bad type');  
console.log(  
  err instanceof TypeError,  
  err instanceof Error,  
  err instanceof RangeError  
)
```

PREDICT OUTPUT

DAY 4

HARD

61

pipe(...fns) function banao

Pehle function ka output doosra ka input.
Koi bhi step throw kare → PipelineError with step index.
Async functions bhi support karo.

CODE KARO

DAY 4

HARD

62

tryCatch(fn, fallback) utility banao

fn throw kare → fallback value return ho.
Sync + async dono handle kare.
fallback khud ek function bhi ho sake.

CODE KARO

DAY 4

63

Singleton config module banao

Config sirf ek baar initialize ho.
Multiple imports mein exact same instance mile.
Test karo: reference equality check karo.

CODE KARO

DAY 4

HARD

64

Named + default export ek file mein

Doosri file mein dono ek hi import line mein lao.
Syntax exactly kya hoga? Edge cases bhi batao.

CODE KARO

DAY 4

65

SafeMap class banao

get(key, defaultVal) – key missing → default return, no error.
getOrThrow(key) – key missing → custom KeyNotFoundError.
has(key), set(key, val), delete(key) bhi implement karo.

CODE KARO

DAY 4

HARD

71

Result monad banao

Result.ok(value) aur Result.err(error)
map(fn), flatMap(fn), getOrElse(default), isOk()
Bina try/catch ke chained error handling demo karo.

CODE KARO

DAY 4

HARD

72

Circular dependency problem demo karo

a.js imports b.js, b.js imports a.js.
Kya hoga at runtime? Kaun undefined milega?
Solve kaise karoge – refactor dikhao.

CODE KARO

DAY 4

73

Kva print hoga — order bhi batao:

```
const p = new Promise((res, rej) => {  
  try { throw new Error('sync') }  
  catch(e) { rej(e) }  
});  
p.catch(e => console.log('caught:', e.message));
```

PREDICT OUTPUT

DAY 4

HARD

76

Module Explorer

User JS code paste kare → count karo:

- try/catch blocks
- import statements
- export statements

Regex use karo — no AST parser.

MINI-PROJECT

DAY 4

77

Error Dashboard

window.onerror aur unhandledrejection capture karo.

Har error ke saath: type, message, timestamp, stack.

Live history DOM mein dikhao — clear button bhi.

MINI-PROJECT

DAY 4

HARD

80

Plugin System

PluginManager class:

- register(name, module)
- run(name, ...args)
- unregister(name)

Plugin nahi mila → PluginNotFoundError. 3 plugins demo karo.

MINI-PROJECT

DAY 4

HARD

83

withErrorLoading(fn) decorator banao

Any function wrap kare.

Error aane par log karo: { fnName, args, error, timestamp }.

Original function ka return value preserve karo.

CODE KARO

DAY 4

85

Kva print hoga?

```
function throws() { throw new RangeError('out') }
try { throws() }
catch(e) {
  if (e instanceof TypeError) throw e;
  console.log('range caught');
}
```

PREDICT OUTPUT

DAY 4

88

AsvncOueue class banao

Tasks ek ek karke chalein – concurrency = 1.
add(asyncFn) se queue mein dalo.
Task fail ho → queue mat ruko – errors collect karo, baaki chalte rahe.

CODE KARO

DAY 4

HARD

89

toString() ka exact output kva hoga?

```
const e = new Error();
e.name = 'Custom';
e.message = 'msg';
console.log(`${e}`); // Template literal – kya hoga?
```

PREDICT OUTPUT

DAY 4

HARD

91

Module Bundler Concept Demo

a.js imports b.js, b.js imports c.js
Manually execution order resolve karo.
Console.log sequence se dependency graph visualize karo.

MINI-PROJECT

DAY 4

HARD

94

Error Recovery UI

Koi bhi action fail ho → user ko friendly error + Retry button.
global window.onerror handler.
Errors ka count dikhao – 'Try again' flow complete karo.

MINI-PROJECT

DAY 4

96

typecheck(schema. data) runtime validator banao

Schema: { name: 'string', age: 'number', tags: 'string[]' }
Mismatch par TypeMismatchError throw karo with:

- field name
- expected type
- actual type

CODE KARO

DAY 4

HARD

99

Chainable Validator banao

```
Validator.string()  
.minLength(3)  
.maxLength(50)  
.matches(/^[a-z]+$/)  
.validate('hello')  
Har rule fail hone par ValidationError with rule name.
```

CODE KARO

DAY 4

HARD

Day 5 — DOM Manipulation & Events[querySelector](#) · [createElement](#) · [Event Delegation](#) · [LocalStorage](#) · [Observers](#)

26

Bubblina order mein output kva hoga?

```
// <div id='parent'><span id='child'>click</span></div>  
// Dono pe click listener (bubbling)  
// Span click karne par?
```

PREDICT OUTPUT

DAY 5

27

Kva aavega?

```
const el = document.createElement('div');  
el.textContent = '<b>bold</b>';  
console.log(el.innerHTML)
```

PREDICT OUTPUT

DAY 5

28

Dono ka fark explain karo:

```
e.stopPropagation() // kya rokta hai?  
e.preventDefault() // kya rokta hai?  
// Dono saath hoon – kya hoga?
```

PREDICT OUTPUT

DAY 5

HARD

29

Kva return karega aur kvun?

```
localStorage.setItem('x', { a: 1 });  
console.log(localStorage.getItem('x'));
```

PREDICT OUTPUT

DAY 5

30

Kva print hoga?

```
const ul = document.createElement('ul');
ul.innerHTML = '<li>A</li><li>B</li>';
const items = ul.querySelectorAll('li');
items.forEach(i => i.remove());
console.log(ul.children.length)
```

PREDICT OUTPUT

DAY 5

HARD

31

Kva difference hai return value mein?

```
document.getElementById('myId')
document.querySelector('#myId')
// Functionally same? Kab farq padta hai?
```

PREDICT OUTPUT

DAY 5

32

Kva print hoga?

```
const p = document.createElement('p');
document.body.appendChild(p);
p.addEventListener('click', () => console.log('fired'));
p.dispatchEvent(new Event('click'));
p.remove();
p.dispatchEvent(new Event('click'));
```

PREDICT OUTPUT

DAY 5

HARD

33

Dono events ka behavior batao:

```
// input pe input + change dono listeners hain
// 1) User type kare → kaun fire hoga?
// 2) User Tab dabaye → kaun fire hoga?
```

PREDICT OUTPUT

DAY 5

34

e.target vs e.currentTarget — fark kva hai?

```
// Parent pe listener, child pe click hua
// e.target = ?
// e.currentTarget = ?
// Concrete example ke saath explain karo.
```

PREDICT OUTPUT

DAY 5

HARD

35

Kva data milega — dono cases mein?

```
sessionStorage.setItem('key', 'val');  
// 1) Same tab, page reload karo  
// 2) Naya tab kholo  
// Dono mein milega ya nahi?
```

PREDICT OUTPUT

DAY 5

36

Event delegation implement karo

ul pe sirf ek listener. Kisi bhi li click par text log ho.
Dynamically add hone wale li bhi handle hon – no rebinding.

CODE KARO

DAY 5

37

DOM manipulation — vanilla JS only

querySelector se paragraph ka text change karo.
Background color bhi change karo. Zero jQuery/libraries.

CODE KARO

DAY 5

38

virtualDOM(config) function banao

Input: { tag: 'div', props: { id: 'x', class: 'box' },
children: [{ tag: 'span', text: 'hi' }] }
Output: Real DOM node – recursively build karo.

CODE KARO

DAY 5

HARD

39

Form validation karo — vanilla JS

Submit default rokko. Name + email validate karo JS mein.
Error messages DOM mein proper jagah dikhao.

CODE KARO

DAY 5

40

Drag & Drop reorder banao

dragstart, dragover, drop events use karo.
Final order localStorage mein save karo.
Page reload par order restore ho.

CODE KARO

DAY 5

HARD

41

localStorage wrapper class banao

get(key), set(key, val), remove(key), clear() – sab JSON serialize kare.
Bonus: set(key, val, ttlSeconds) – expiry support bhi do.

CODE KARO

DAY 5

42

Do counters banao

1) sessionStorage counter – reload par reset
2) localStorage counter – reload par persist
Dono UI mein clearly alag dikhao.

CODE KARO

DAY 5

43

MutationObserver se selective DOM loader banao

Naye nodes add hone par log karo sirf agar data-track attribute ho.
Cleanup bhi karo – observer disconnect karo jab zaroorat na ho.

CODE KARO

DAY 5

HARD

44

Image gallery banao — zero CSS frameworks

Thumbnails click par large preview dikhao.
Next / Prev navigation buttons bhi.
Keyboard arrows bhi support karo.

CODE KARO

DAY 5

45

PubSub module banao using closures

subscribe(event, fn), publish(event, data), unsubscribe(event, fn)
DOM events pe nahi – internal event bus.
Memory leak avoid karo – proper unsubscribe test karo.

CODE KARO

DAY 5

HARD

46

DOM traversal utility banao

getSiblings(el) – saare siblings
getAncestors(el) – parent chain
getAllDescendants(el) – sab children recursively

CODE KARO

DAY 5

47

debounce aur throttle dono banao

debounce(fn, ms) – window resize pe lagao, 300ms baad fire ho.
throttle(fn, ms) – mousemove pe lagao.
Dono ka behavior side by side demo karo.

CODE KARO

DAY 5

HARD

48

Keyboard shortcut system banao

Ctrl+S → 'Saved!', Ctrl+Z → 'Undo!', Escape → 'Cancelled'
keydown event use karo.
Shortcuts easily add/remove ho sakein.

CODE KARO

DAY 5

49

Lazv image loader banao

IntersectionObserver use karo.
data-src → src sirf jab element viewport mein aaye.
Placeholder/blur pehle dikhe, real image load hone ke baad swap karo.

CODE KARO

DAY 5

HARD

50

Dnamic table banao proogrammaticallv

createElement + appendChild se – no innerHTML.
Data: array of { name, age, city }.
Header row bhi dynamically banao.

CODE KARO

DAY 5

56

Kva print hoga?

```
const div = document.createElement('div');  
div.innerHTML = 'hi <scr' + 'ipt>alert(1)</scr' + 'ipt>';  
console.log(div.textContent)
```

PREDICT OUTPUT

DAY 5

57

Canturina vs Bubbling — exact order kva hoga?

```
el.addEventListener('click', fn1, true) // capturing  
el.addEventListener('click', fn2, false) // bubbling  
// Nested elements: parent → child click par sequence?
```

PREDICT OUTPUT

DAY 5

HARD

58

cloneNode behavior batao:

```
node.cloneNode(true) // deep clone  
node.cloneNode(false) // shallow clone  
// Event listeners clone honge kya?
```

PREDICT OUTPUT

DAY 5

59

DocumentFragment kyun use karte hain?

```
// Option A: 100 baar seedha appendChild()  
// Option B: Fragment mein add karke ek baar append  
// Reflow/repaint mein kya farq padta hai?
```

PREDICT OUTPUT

DAY 5

HARD

60

Kab kaun best practice hai?

```
el.style.color = 'red'  
el.setAttribute('style', 'color:red')  
el.classList.add('text-red')  
// Teen approaches – teen scenarios batao.
```

PREDICT OUTPUT

DAY 5

66

Accordion component banao

Click par section expand/collapse.
Ek waqt mein sirf ek section open.
Smooth CSS transition – zero JS animation libraries.

CODE KARO

DAY 5

67

Modal class banao

new Modal({ title, content, onClose }) → DOM mein inject ho.
Close: Escape key, backdrop click, X button.
Focus trap: Tab key modal ke andar hi ghume.

CODE KARO

DAY 5

HARD

68

Infinite scroll banao

Bottom se 150px pehle data fetch ho.
Loading spinner dikhao during fetch.
IntersectionObserver ya scroll event – apna choice justify karo.

CODE KARO

DAY 5

69

Client-side Router banao — History API

```
navigate(path), back(), on(path, handler)  
Browser back/forward button handle karo (popstate).  
No page reload. Route matching with params support karo.
```

CODE KARO

DAY 5

HARD

70

DOM diffing function banao

diff(oldVNode, newVNode)
Virtual trees compare karo, sirf changed parts update karo.
Real DOM pe minimal patch apply karo.

CODE KARO

DAY 5

HARD

74

Kya hoga?

```
const a = document.createElement('a');  
a.href = 'javascript:void(0)';  
a.click();  
console.log('after click')
```

PREDICT OUTPUT

DAY 5

HARD

75

Toast notification system banao

toast.show(message, type, durationMs)
Types: success / error / warning (alag colors)
Auto-dismiss + manual close. Multiple toasts stack hon.

CODE KARO

DAY 5

78

Vanilla JS To-Do App

- Add, delete, mark done, edit in-place
- Filter: All / Active / Done
- localStorage persist – reload par wapas aaye
- Zero libraries – zero frameworks

MINI-PROJECT

DAY 5

79

Live DOM Inspector

Kisi bhi element pe hover karo toh floating panel mein dikhao:
tag name, id, class list, innerHTML length
Element ko visually highlight bhi karo (outline).

MINI-PROJECT

DAY 5

HARD

81

Custom EventTarget polyfill banao

Bina browser API ke likho:
add/removeEventListener, dispatchEvent
Custom Event objects with detail property support karo.

CODE KARO

DAY 5

HARD

82

Live search with text highlight banao

Input type karo → list realtime filter ho.
Matching characters highlight (bold/color) karo.
300ms debounce lagao.

CODE KARO

DAY 5

84

Kva print hoga?

```
const div = document.createElement('div');
div.addEventListener('click', () => console.log(1));
const clone = div.cloneNode(true);
document.body.append(clone);
clone.click()
```

PREDICT OUTPUT

DAY 5

HARD

86

SortableTable class banao

Column header click par ascending/descending sort toggle.
Data array se render ho.
Active sort column pe direction arrow dikhao.

CODE KARO

DAY 5

HARD

87

Clipboard copy button banao

'Copy to clipboard' click karo → text copy ho.
Button text: 'Copied! ✓' – 2 sec baad wapas.
Permission deny ho toh graceful fallback.

CODE KARO

DAY 5

90

Form auto-save feature banao

User type kare → 1 sec debounce → localStorage save.
Page aane par draft restore ho.
'Draft saved at 14:32' indicator DOM mein dikhao.

CODE KARO

DAY 5

HARD

92

Image Carousel — vanilla JS

- Auto-play every 3 seconds
- Prev / Next buttons
- Dot indicators (clickable)
- Keyboard arrow key support
- localStorage mein current slide save

MINI-PROJECT

DAY 5

93

Virtual Scroll — 10.000 items

Sirf visible items DOM mein render hon.
 Scroll karte waqt DOM update ho — no lag.
 DevTools Performance tab se verify karo.

MINI-PROJECT

DAY 5

HARD

95

DOM Time Machine

Har 5 seconds mein DOM snapshot lo (outerHTML).
 Undo button se pichla snapshot restore ho.
 Max 10 snapshots — oldest automatically hat jaaye.

MINI-PROJECT

DAY 5

HARD

97

ResizeObserver se element monitor karo

Element ki width/height observe karo.
 Change hone par DOM mein live dimensions update karo.
 Element remove hone par observer disconnect karo.

CODE KARO

DAY 5

98

Kva print hoqa aur kvun?

```
document.body.innerHTML = '';
document.body.innerHTML = '<div id=x></div>';
const el = document.getElementById('x');
document.body.innerHTML = '<div id=x></div>';
el.textContent = 'hi';
console.log(document.getElementById('x').textContent)
```

PREDICT OUTPUT

DAY 5

HARD

Final Boss

Full integration — both days combined. Zero libraries.

100

FINAL BOSS — Vanilla JS Notes App

- Notes CRUD: add, edit, delete, view
- Tags system (multi-tag)
- Search with live highlight
- localStorage persistence
- Global error boundary
- Event delegation throughout
- Min 5 separate JS modules
- Zero libraries. Zero frameworks.

MINI-PROJECT

DAY 4+5

HARD